

TaskFlow CM

Diccionario de Clases

Sprint 2 · 2026

Documentacion tecnica del sistema TaskFlow generada a partir del codigo fuente Java. Describe todas las clases del proyecto organizadas por capa: modelo de dominio, DTOs, excepciones, repositorios JPA, servicios y controladores REST. Incluye descripcion de cada clase, sus campos y sus metodos con parametros y valor de retorno.

Paquete	Clases
Modelo de Dominio	Cuestionario, Logro, LogroUsuario, Pregunta, Simulacion, SimulacionItem, Tarea, Teoria, Usuario
Objetos de Transferencia de Datos (DTO)	LoginRequest, RegisterRequest, TareaRequest, UsuarioResponse
Manejo de Excepciones	BadRequestException, ConflictException, GlobalExceptionHandler, ResourceNotFoundException
Repositorios JPA	CuestionarioRepository, LogroRepository, LogroUsuarioRepository, PreguntaRepository, SimulacionItemR
Servicios	AuthService, GamificacionService, QuizService, ScormService, SimulacionService, TareaService
Controladores REST	AuthController, GamificacionController, HomeController, QuizController, ScormController, SimulacionCont
Configuracion	AppConfig
Aplicacion Principal	TaskflowApplication

Modelo de Dominio

Class Cuestionario @Entity @Data → tabla: cuestionarios

Entidad que representa un cuestionario de evaluacion dentro del modulo educativo de TaskFlow. Un cuestionario agrupa un conjunto de Preguntas y puede ser de tipo "quiz" o "evaluacion". Tiene asociado un nivel de dificultad ("basico" o "avanzado") y material teorico previo (Teoria). Mapeada a la tabla cuestionarios en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa un cuestionario de evaluacion dentro del modulo educativo de TaskFlow. Un cuestionario agrupa un
titulo	Titulo del cuestionario. Campo obligatorio, maximo 200 caracteres.
descripcion	Descripcion general o instrucciones para el estudiante antes de iniciar.
duracionMinutos	Tiempo maximo permitido para completar el cuestionario, expresado en minutos.
tipo	Tipo de evaluacion. Valores posibles: "quiz" o "evaluacion". Campo obligatorio.
nivel	Nivel de dificultad del cuestionario. Valores: "basico" o "avanzado".
createdAt	Fecha y hora en que fue creado el cuestionario.
preguntas	Lista de preguntas asociadas a este cuestionario. Relacion bidireccional OneToMany. Excluida de la serializacion JSON para

Class Logro @Entity @Data → tabla: logros

Entidad que define un logro del sistema de gamificacion de TaskFlow. Los logros son globales y se definen una sola vez en la base de datos. La obtencion de un logro por parte de un usuario especifico se registra en la entidad LogroUsuario. Codigos de logro disponibles: PRIMERA_TAREA, CINCO_TAREAS, DIEZ_TAREAS, PRIMER_QUIZ, TODOS_BASICOS, TODOS_AVANZADO, NIVEL_AVANZADO. Mapeada a la tabla logros en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que define un logro del sistema de gamificacion de TaskFlow. Los logros son globales y se definen una sola vez en
codigo	Identificador unico de negocio del logro. Ejemplos: "PRIMERA_TAREA", "NIVEL_AVANZADO". Usado internamente por Gar
nombre	Nombre legible del logro mostrado al usuario en la interfaz.
descripcion	Descripcion del criterio necesario para obtener el logro.
icono	Emoji o codigo de icono representativo que se muestra junto al logro en la UI.
puntosRecompensa	Puntos adicionales que se suman al total del usuario al obtener este logro.

Class LogroUsuario @Entity @Data → tabla: logros_usuario

Entidad de union que registra los logros obtenidos por cada usuario. Actua como tabla intermedia entre Usuario y Logro. Cada registro representa la obtencion de un logro especifico por un usuario en un momento determinado. Es la unica entidad con una relacion @ManyToOne JPA explicita hacia Logro. Mapeada a la tabla logros_usuario en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad de union que registra los logros obtenidos por cada usuario. Actua como tabla intermedia entre Usuario y Logro. Ca

Campo	Descripcion
usuarioid	Identificador del usuario que obtuvo el logro (FK logica a usuarios.id).
logro	Logro obtenido por el usuario. Relacion ManyToOne hacia la tabla logros.
obtenidoAt	Fecha y hora exacta en que el usuario obtuvo el logro.

Class Pregunta @Entity @Data → tabla: preguntas

Entidad que representa una pregunta individual dentro de un Cuestionario. Soporta dos tipos de pregunta: opcion multiple ("multiple") y respuesta abierta ("abierta"). La validacion de respuestas se realiza de forma insensible a mayusculas y con eliminacion de espacios en blanco (trim). Mapeada a la tabla preguntas en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa una pregunta individual dentro de un Cuestionario. Soporta dos tipos de pregunta: opcion multiple ("multiple") y respuesta abierta ("abierta").
texto	Enunciado completo de la pregunta. Campo obligatorio almacenado como TEXT.
opciones	Opciones de respuesta para preguntas de tipo multiple. Las opciones se separan con el caracter .
respuestaCorrecta	Respuesta correcta de la pregunta. Campo obligatorio. La comparacion se realiza con equalsIgnoreCase(respuesta.trim()).
tipoPregunta	Tipo de pregunta. Valores posibles: "multiple" o "abierta".
puntaje	Puntos que se otorgan al usuario al responder correctamente esta pregunta.
explicacion	Texto de retroalimentacion mostrado al usuario tras responder, independientemente de si la respuesta fue correcta o no.
cuestionario	Cuestionario al que pertenece esta pregunta. Relacion ManyToOne; excluida del JSON para evitar referencias circulares.

Class Simulacion @Entity @Data → tabla: simulaciones

Entidad que representa una sesion de sprint planning simulado en TaskFlow. Cada usuario puede tener como maximo una simulacion en estado "activo" a la vez. Al iniciar una nueva, la simulacion activa previa cambia automaticamente a "abandonado". Una vez ejecutado el sprint y evaluados los resultados, pasa a estado "cerrado". Mapeada a la tabla simulaciones en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa una sesion de sprint planning simulado en TaskFlow. Cada usuario puede tener como maximo una simulacion en estado "activo" a la vez.
usuarioid	Identificador del usuario que ejecuta la simulacion (FK logica a usuarios.id).
estado	Estado actual de la simulacion. Valores posibles: "activo", "cerrado", "abandonado".
sprintGoal	Objetivo del sprint definido por el usuario al iniciar la simulacion. Serializado en JSON como "metaSprint" mediante @JsonPr...
velocidadEquipo	Velocidad del equipo expresada en story points por sprint. Sirve como referencia para evaluar si el sprint fue exitoso.
createdAt	Fecha y hora en que se inicio la simulacion.
cerradoAt	Fecha y hora en que se cerro el sprint. Es null mientras la simulacion este activa.

Class SimulacionItem @Entity @Data → tabla: simulacion_items

Entidad que representa un item del backlog dentro de una Simulacion. Al iniciar cada simulacion, el sistema genera automaticamente 12 items predefinidos con historias de usuario reales de metodologia Scrum. El usuario puede seleccionar cuales incluir en el sprint (enSprint = true) y luego marcarlos como completados durante la ejecucion. Mapeada a la tabla simulacion_items en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa un item del backlog dentro de una Simulacion. Al iniciar cada simulacion, el sistema genera automati
simulacionId	Identificador de la simulacion a la que pertenece este item (FK logica a simulaciones.id).
titulo	Nombre de la historia de usuario o tarea del backlog simulado.
storyPoints	Estimacion del esfuerzo requerido para completar este item, en story points.
enSprint	Indica si el item fue seleccionado para ser ejecutado en el sprint actual. false por defecto; cambia a true mediante el endpoi
completado	Indica si el item fue marcado como entregado durante la ejecucion del sprint. false por defecto.
orden	Posicion de visualizacion del item dentro del backlog (orden ascendente).

Class Tarea @Entity @Data → tabla: tareas

Entidad que representa una tarea dentro del tablero Kanban de TaskFlow. Cada tarea pertenece a un usuario y puede encontrarse en uno de tres estados: pendiente, en_proceso o completada. Al completarse, el sistema de gamificacion otorga puntos al usuario propietario. Mapeada a la tabla tareas en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa una tarea dentro del tablero Kanban de TaskFlow. Cada tarea pertenece a un usuario y puede encon
titulo	Titulo descriptivo de la tarea. Campo obligatorio (NOT NULL).
descripcion	Descripcion detallada opcional con informacion adicional sobre la tarea.
columnaId	Columna del tablero Kanban donde se ubica la tarea. Valores: 0=Backlog, 1=To Do, 2=In Progress, 3=Done.
prioridad	Nivel de prioridad de la tarea en escala del 1 (alta) al 10 (baja). Valor por defecto: 5.
fechaVencimiento	Fecha limite de entrega de la tarea. Opcional.
estado	Estado actual de la tarea. Valores validos: "pendiente" (defecto), "en_proceso", "completada".
usuarioid	Identificador del usuario propietario de la tarea (FK logica a usuarios.id).
puntosHistoria	Estimacion del esfuerzo requerido expresada en story points. Valor por defecto: 3.
createdAt	Fecha y hora en que fue creada la tarea.

Class Teoria @Entity @Data → tabla: teorias

Entidad que representa un bloque de contenido teorico asociado a un Cuestionario. El material teorico se presenta al estudiante antes de responder el quiz, ordenado segun el campo orden. Puede contener texto plano o HTML. Mapeada a la tabla teorias en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa un bloque de contenido teorico asociado a un Cuestionario. El material teorico se presenta al estudiante.
cuestionarioid	Identificador del cuestionario al que pertenece este bloque teorico (FK logica a cuestionarios.id).
titulo	Titulo del bloque teorico mostrado como encabezado al estudiante.
contenido	Contenido del material teorico. Puede contener texto plano o HTML enriquecido.
orden	Posicion de este bloque dentro del material teorico del cuestionario (orden ascendente).
fuelle	Referencia bibliografica, URL o fuente de donde proviene el contenido teorico.

Class Usuario @Entity @Data → tabla: usuarios

Entidad que representa a un usuario registrado en el sistema TaskFlow. Almacena las credenciales de autenticacion, el nivel de avance educativo (basico/avanzado) y los puntos acumulados por el sistema de gamificacion. Mapeada a la tabla usuarios en PostgreSQL.

Campos:

Campo	Descripcion
id	Entidad que representa a un usuario registrado en el sistema TaskFlow. Almacena las credenciales de autenticacion, el nivel de acceso educativo y los puntos acumulados.
nombre	Nombre completo del usuario tal como fue ingresado en el registro.
email	Correo electronico del usuario. Debe ser unico en el sistema. Se almacena en minusculas para garantizar unicidad sin distincion de mayusculas.
password	Contraseña del usuario cifrada con BCrypt. Nunca se almacena en texto plano.
nivel	Nivel de acceso educativo del usuario. Valores posibles: "basico" (por defecto) o "avanzado". Cambia automaticamente cuando se completan 4 quizzes.
quizzesCompletados	Cantidad de quizzes completados por el usuario. Cuando alcanza 4, el nivel asciende automaticamente a "avanzado".
createdAt	Fecha y hora en que el usuario se registro en el sistema.
puntos	Puntos acumulados por el usuario a traves del sistema de gamificacion. Se suman puntos al crear tareas, completarlas y aprobarlas.

Objetos de Transferencia de Datos (DTO)

Class LoginRequest @Data

DTO que encapsula las credenciales de autentificacion del usuario. Utilizado en el endpoint POST /login. Validado con @Valid; si las credenciales son incorrectas, AuthService lanza BadRequestException con HTTP 400.

Campos:

Campo	Descripcion
email	DTO que encapsula las credenciales de autentificacion del usuario. Utilizado en el endpoint POST /login. Validado con @Valid
password	Contraseña en texto plano para verificar contra el hash BCrypt almacenado.

Class RegisterRequest @Data

DTO que encapsula los datos necesarios para registrar un nuevo usuario. Validado automaticamente por Spring mediante @Valid en el controlador. Si alguna restriccion no se cumple, se lanza MethodArgumentNotValidException y el GlobalExceptionHandler responde con HTTP 400.

Campos:

Campo	Descripcion
nombre	DTO que encapsula los datos necesarios para registrar un nuevo usuario. Validado automaticamente por Spring mediante @Valid
email	Correo electronico del usuario. Obligatorio y debe tener formato valido. Se almacena en minusculas en el sistema.
password	Contraseña en texto plano. Obligatoria, minimo 8 caracteres. Adicionalmente validada por AuthService.validarPassword(): de

Class TareaRequest @Data

DTO que encapsula los datos para crear o editar una tarea en el tablero Kanban. Validado automaticamente por Spring mediante @Valid. Utilizado en los endpoints POST /tareas y PUT /tareas/{id}.

Campos:

Campo	Descripcion
titulo	DTO que encapsula los datos para crear o editar una tarea en el tablero Kanban. Validado automaticamente por Spring mediante @Valid
descripcion	Descripcion detallada opcional. Maximo 1000 caracteres.
usuarioid	ID del usuario propietario de la tarea. Obligatorio.
prioridad	Nivel de prioridad entre 1 (alta) y 10 (baja). Opcional; defecto 5.
fechaVencimiento	Fecha limite de entrega de la tarea. Opcional.
puntosHistoria	Estimacion de esfuerzo en story points. Opcional; defecto 3.

Class UsuarioResponse @Data

DTO de respuesta que expone los datos publicos de un usuario. Excluye deliberadamente el campo password para evitar su exposicion en las respuestas HTTP. Retornado por los endpoints de registro, login y completar quiz.

Campos:

Campo	Descripcion
id	DTO de respuesta que expone los datos publicos de un usuario. Excluye deliberadamente el campo password para evitar su
nombre	Nombre completo del usuario.
email	Correo electronico del usuario.
nivel	Nivel educativo actual: "basico" o "avanzado".
quizzesCompletados	Cantidad de quizzes completados hasta el momento.
puntos	Puntos acumulados por gamificacion.
createdAt	Fecha y hora de registro del usuario.

Metodos:

Metodo	Descripcion	Retorna
from()	DTO de respuesta que expone los datos publicos de un usuario. Excluye deliberadamente el campo password para evitar su	DTO de respuesta que expone los datos publicos de un usuario. Excluye deliberadamente el campo password para evitar su

Manejo de Excepciones

Class BadRequestException

Excepcion que representa un error de solicitud invalida (HTTP 400 Bad Request). Se lanza cuando los datos enviados por el cliente son semanticamente incorrectos, por ejemplo: credenciales incorrectas, estado de tarea invalido o contrasena que no cumple los requisitos de seguridad. Capturada y manejada por GlobalExceptionHandler.

Class ConflictException

Excepcion que representa un conflicto de datos (HTTP 409 Conflict). Se lanza cuando una operacion viola una restriccion de unicidad, por ejemplo al intentar registrar un email que ya existe en el sistema. Capturada y manejada por GlobalExceptionHandler.

Class GlobalExceptionHandler @RestController @RestControllerAdvice

Manejador global de excepciones para todos los controladores REST de TaskFlow. Intercepta las excepciones lanzadas en cualquier capa del sistema y las convierte en respuestas HTTP estructuradas con timestamp, codigo de estado y mensaje de error. Utiliza @RestControllerAdvice de Spring para aplicarse globalmente.

Metodos:

Metodo	Descripcion	Retorna
handleNotFound()	Manejador global de excepciones para todos los controladores REST de TaskFlow cuando se lanzan excepciones de tipo NotFoundException.	HTTP 404 y mensaje de error.
handleBadRequest()	Maneja solicitudes invalidas. Retorna HTTP 400. @param ex excepcion lanzada por el cliente y mensaje de estado.	HTTP 400 y mensaje de estado.
handleConflict()	Maneja conflictos de datos. Retorna HTTP 409. @param ex excepcion lanzada por el servidor y mensaje de estado.	HTTP 409 y mensaje de estado.
handleValidation()	Maneja errores de validacion de Bean Validation (@Valid). Retorna HTTP 400. @param ex excepcion lanzada por el cliente y mensaje de estado.	HTTP 400 y mensaje de estado.
handleGeneric()	Maneja cualquier excepcion no contemplada. Retorna HTTP 500. Registra el error en el log del servidor. @param ex excepcion lanzada por el servidor.	HTTP 500 y mensaje de estado.
buildError()	Construye el cuerpo estandar de respuesta de error con timestamp, status code y mensaje de error.	ResponseEntity con el cuerpo de error.

Class ResourceNotFoundException

Excepcion que representa un recurso no encontrado (HTTP 404 Not Found). Se lanza cuando se intenta acceder o modificar una entidad que no existe en la base de datos, por ejemplo una tarea, usuario o cuestionario con un ID inexistente. Capturada y manejada por GlobalExceptionHandler.

Repositorios JPA

Interface CuestionarioRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Cuestionario`. Provee metodos para filtrar cuestionarios por nivel de dificultad y tipo.

Interface LogroRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Logro`. Utilizado por `GamificacionService` para buscar logros por su codigo de negocio antes de otorgarlos a un usuario.

Interface LogroUsuarioRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.LogroUsuario`. Gestiona los registros de logros obtenidos por cada usuario. Permite consultar los logros de un usuario y verificar si ya obtuvo un logro especifico.

Interface PreguntaRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Pregunta`. Permite obtener todas las preguntas de un cuestionario dado, utilizado por `QuizService` al cargar un quiz para el estudiante.

Interface SimulacionItemRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.SimulacionItem`. Gestiona los items del backlog de cada simulacion. Provee consultas para obtener todos los items o solo los seleccionados para el sprint.

Interface SimulacionRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Simulacion`. Utilizado por `SimulacionService` para gestionar el ciclo de vida de las simulaciones de sprint. La consulta mas critica es la de simulacion activa por usuario, que garantiza que solo exista una a la vez.

Interface TareaRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Tarea`. Extiende `JpaRepository` con consultas personalizadas para el tablero Kanban y el sistema de gamificacion. Incluye una consulta JPQL para agrupacion por estado.

Interface TeoriaRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Teoria`. Permite obtener el material teorico de un cuestionario ordenado para presentarlo al estudiante en el orden correcto antes del quiz.

Interface UsuarioRepository

Repositorio JPA para la entidad `com.ingenieria.taskflow.model.Usuario`. Extiende `JpaRepository` para heredar operaciones CRUD estandar. Provee metodos de busqueda por email utilizados en autenticacion y registro.

Servicios

Class AuthService @Service

Servicio de autentificacion y gestion de usuarios de TaskFlow. Maneja el registro de nuevos usuarios, el inicio de sesion y el avance educativo al completar quizzes. Integra cifrado BCrypt para contraseñas y delega en GamificacionService el otorgamiento de puntos y logros.

Metodos:

Metodo	Descripcion	Retorna
registrar()	Servicio de autentificacion y gestion de usuarios de TaskFlow. Maneja el registro de nuevos usuarios, el inicio de sesion y el avance educativo al completar quizzes. Integra cifrado BCrypt para contraseñas y delega en GamificacionService el otorgamiento de puntos y logros.	UsuarioResponse con los datos del usuario registrado
login()	Autentica un usuario verificando sus credenciales contra el hash BCrypt	UsuarioResponse con los datos del usuario autenticado
completarQuiz()	Registra la finalizacion de un quiz por parte del usuario. Incrementa el puntaje del usuario y otorga puntos y logros segun los umbrales definidos.	UsuarioResponse con los datos actualizados del usuario
validarPassword()	Valida que la contraseña cumpla los requisitos minimos de seguridad: minimo 8 caracteres, al menos una mayuscula, un numero y un caracter especial.	Booleano indicando si es valido o no

Class GamificacionService @Service

Servicio que implementa el sistema de gamificacion de TaskFlow. Procesa actividades del usuario (crear tareas, completarlas, aprobar quizzes, subir de nivel) y otorga puntos y logros segun los umbrales definidos. Los tipos de actividad soportados son: TAREA_CREADA (+10 pts), TAREA_COMPLETADA (+20 pts), QUIZ_COMPLETADO (+50 pts), NIVEL_AVANZADO (otorga logro, sin puntos adicionales directos).

Metodos:

Metodo	Descripcion	Retorna
procesarActividad()	Servicio que implementa el sistema de gamificacion de TaskFlow. Procesa actividades del usuario (crear tareas, completarlas, aprobar quizzes, subir de nivel) y otorga puntos y logros segun los umbrales definidos.	UsuarioResponse con los datos actualizados del usuario
otorgarLogro()	Intenta otorgar un logro al usuario si aun no lo tiene. Si el logro no existe en la base de datos, se crea y se otorga.	Booleano indicando si se otorgo o no
obtenerLogrosUsuario()	Obtiene todos los logros obtenidos por un usuario especifico. @param usuarioId: ID del usuario	Lista de logros obtenidos
obtenerTodosLogros()	Obtiene la definicion completa de todos los logros disponibles en el sistema	Lista de todos los logros disponibles

Class QuizService @Service

Servicio que gestiona el modulo educativo de quizzes y teoria de TaskFlow. Provee cuestionarios, preguntas y material teorico, ademas de validar las respuestas del estudiante de forma insensible a mayusculas y espacios.

Metodos:

Metodo	Descripcion	Retorna
obtenerTodos()	Servicio que gestiona el modulo educativo de quizzes y teoria de TaskFlow. Provee cuestionarios, preguntas y material teorico, ademas de validar las respuestas del estudiante de forma insensible a mayusculas y espacios.	Lista de todos los cuestionarios, preguntas y material teorico
obtenerPorNivel()	Retorna los cuestionarios filtrados por nivel de dificultad. @param nivel: Nivel de dificultad	Lista de cuestionarios filtrados
obtenerPreguntasPorCuestionario()	Retorna todas las preguntas de un cuestionario especifico. @param cuestionarioId: ID del cuestionario	Lista de preguntas del cuestionario
obtenerTeoriaPorCuestionario()	Retorna el material teorico de un cuestionario ordenado para su presentacion	Lista de material teorico del cuestionario
validarRespuesta()	Valida la respuesta del estudiante para una pregunta especifica. La comparacion es insensible a mayusculas y espacios.	Booleano indicando si es correcta o no

Class ScormService @Service

Campos:

Campo	Descripcion
BACKEND_URL	URL pública del backend — los JS del paquete SCORM apuntan aquí

Class SimulacionService @Service

Servicio que gestiona la simulacion de sprint planning de TaskFlow. Permite al estudiante practicar la planificacion de un sprint Scrum con 12 historias de usuario predefinidas del mundo real. Calcula metricas al cerrar el sprint: puntos planeados vs entregados, velocidad real, exito (entregados >= planeados * 0.8) y genera una retrospectiva automatica.

Metodos:

Metodo	Descripcion	Retorna
iniciarSimulacion()	Servicio que gestiona la simulacion de sprint planning de TaskFlow. Permite al estudiante practicar la planificacion de un sprint Scrum con 12 historias de usuario predefinidas del mundo real.	mapa de estado inicial practica (listo, en progreso, pendiente, completado)
obtenerSimulacionActiva()	Consulta la simulacion activa del usuario si existe. @param usuarioid ID de usuario @return SimulacionItem o null si no hay simulacion activa	SimulacionItem o null
toggleItemSprint()	Alterna el estado de un item entre incluido y excluido del sprint. @param itemid ID de item @return SimulacionItem actualizado	SimulacionItem
completarItem()	Marca un item del sprint como completado (entregado). @param itemid ID de item @return SimulacionItem actualizado	SimulacionItem
cerrarSprint()	Cierra el sprint de una simulacion y calcula los resultados finales. El sprint es exitoso si los puntos entregados >= 80% de los planeados	mapa de estado final practica (listo, en progreso, pendiente, completado)
generarRetrospectiva()	Genera el texto de retrospectiva del sprint segun el resultado obtenido. @param usuarioid ID de usuario @return String	String

Class TareaService @Service

Servicio de logica de negocio para la gestion de tareas del tablero Kanban. Controla el ciclo de vida completo de una tarea: creacion, actualizacion de estado, edicion, eliminacion y obtencion de estadisticas. Dispara eventos de gamificacion al crear (TAREA_CREADA) y completar tareas (TAREA_COMPLETADA). Solo acepta los estados: "pendiente", "en_proceso", "completada".

Metodos:

Metodo	Descripcion	Retorna
obtenerPorUsuario()	Servicio de logica de negocio para la gestion de tareas del tablero Kanban. Controla el ciclo de vida completo de una tarea: creacion, actualizacion de estado, edicion, eliminacion y obtencion de estadisticas.	lista de tareas de un usuario
crear()	Crea una nueva tarea en el tablero Kanban y otorga puntos de gamificacion al crearla. Dispara el evento TAREA_CREADA que suma puntos al usuario.	entidad Tarea creada
actualizarEstado()	Actualiza el estado de una tarea. Si el nuevo estado es "completada", dispara el evento TAREA_COMPLETADA que suma puntos al usuario.	entidad Tarea actualizada
obtenerStats()	Calcula las estadisticas de tareas de un usuario agrupadas por estado. @param usuarioid ID de usuario @return Map	Map
eliminar()	Elimina una tarea de la base de datos verificando su existencia previa. @param id ID de la tarea a eliminar @throws ResourceNotFoundException	Booleano
editar()	Edita los campos de una tarea existente. @param id ID de la tarea a editar @param datos Tarea con los nuevos valores	entidad Tarea actualizada

Controladores REST

Class AuthController @RestController

Controlador REST que expone los endpoints de autenticacion de TaskFlow. Gestiona el registro de nuevos usuarios, el inicio de sesion y el registro de quizzes completados. Todas las respuestas retornan un UsuarioResponse sin exponer la contrasena. POST /usuarios — Registrar nuevo usuario (HTTP 201) POST /login — Iniciar sesion (HTTP 200) POST /usuarios/{id}/completar-quiz — Registrar quiz completado

Metodos:

Metodo	Descripcion	Retorna
registrar()	Controlador REST que expone los endpoints de autenticacion de TaskFlow. Gestiona el registro de nuevos usuarios, el inicio de sesion y el registro de quizzes completados. Todas las respuestas retornan un UsuarioResponse sin exponer la contrasena.	201 OK con UsuarioResponse
login()	Autentica al usuario con email y contrasena. Retorna HTTP 200 con dato de UsuarioResponse si las credenciales validadas son correctas.	200 OK con UsuarioResponse
completarQuiz()	Registra la finalizacion de un quiz y actualiza puntos y nivel del usuario.	200 OK con UsuarioResponse actualizado

Class GamificacionController @RestController

Controlador REST del sistema de gamificacion de TaskFlow. Permite procesar actividades manualmente y consultar los logros globales del sistema o los obtenidos por un usuario especifico. Base: /api/gamificacion POST /api/gamificacion/actividad — Procesar actividad (puntos y logros) GET /api/gamificacion/logros — Todos los logros del sistema GET /api/gamificacion/logros/{usuarioId} — Logros obtenidos por un usuario

Class HomeController @Controller

Controlador MVC que redirige la raiz de la aplicacion a la pagina de login. Al acceder a /, Spring reenvía la peticion a /login.html que se sirve como recurso estatico desde src/main/resources/static/.

Metodos:

Metodo	Descripcion	Retorna
home()	Controlador MVC que redirige la raiz de la aplicacion a la pagina de login. Al acceder a /, Spring reenvía la peticion a /login.html que se sirve como recurso estatico desde src/main/resources/static/.	200 OK con login.html

Class QuizController @RestController

Controlador REST del modulo educativo de quizzes de TaskFlow. Expone endpoints para obtener cuestionarios, teoria y preguntas, validar respuestas y consultar el progreso educativo del usuario. Base: /api GET /api/quiz — Todos los cuestionarios GET /api/quiz/nivel/{nivel} — Cuestionarios por nivel GET /api/quiz/{id} — Preguntas de un cuestionario GET /api/teoria/{id} — Teoria de un cuestionario POST /api/quiz/answer — Validar respuesta GET /api/user/progress — Progreso educativo del usuario

Class ScormController @RestController

Endpoints de exportación SCORM 1.2. GET /api/scorm/app → Descarga la aplicación TaskFlow completa empaquetada como SCORM 1.2. Incluye login.html + index.html + todos los CSS/JS + scorm_init.js. Sube el ZIP a cualquier LMS (Moodle, Canvas, Blackboard, ILIAS...). GET /api/scorm/quiz/{cuestionarioId} → Descarga un cuestionario individual como paquete SCORM 1.2 autocontenido (HTML + teoría + quiz con tracking, sin depender de la API).

Metodos:

Metodo	Descripcion	Retorna
exportarAplicacion()	Endpoints de exportación SCORM 1.2. GET /api/scorm/app → Descarga la aplicación TaskFlow completa empaquetada como archivo zip	-
exportarQuiz()	Exporta un cuestionario individual como paquete SCORM 1.2	-

Class SimulacionController @RestController

Controlador REST del modulo de simulacion de sprint planning de TaskFlow. Gestiona el ciclo completo de una simulacion: inicio, consulta, seleccion de items para el sprint, completado de items y cierre del sprint. Base: /api/simulacion POST /api/simulacion/iniciar — Iniciar nueva simulacion GET /api/simulacion/activa/{usuarioId} — Consultar simulacion activa PATCH /api/simulacion/item/{id/toggle} — Incluir/excluir item del sprint PATCH /api/simulacion/item/{id/completar} — Marcar item como completado POST /api/simulacion/cerrar/{id} — Cerrar sprint y obtener resultados

Class TareaController @RestController

Controlador REST que expone los endpoints del tablero Kanban de TaskFlow. Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario. GET /tareas?usuarioId= — Listar tareas del usuario POST /tareas — Crear nueva tarea (HTTP 201) PATCH /tareas/{id/status} — Actualizar estado PUT /tareas/{id} — Editar tarea completa DELETE /tareas/{id} — Eliminar tarea GET /api/user/stats?usuarioId= — Estadisticas por estado

Metodos:

Metodo	Descripcion	Retorna
obtenerTareas()	Controlador REST que expone los endpoints del tablero Kanban de TaskFlow. Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario. GET /tareas?usuarioId= — Listar tareas del usuario POST /tareas — Crear nueva tarea (HTTP 201) PATCH /tareas/{id/status} — Actualizar estado PUT /tareas/{id} — Editar tarea completa DELETE /tareas/{id} — Eliminar tarea GET /api/user/stats?usuarioId= — Estadisticas por estado	200 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.
crearTarea()	Crea una nueva tarea en el tablero Kanban. Retorna HTTP 201. @param id ID de la tarea a crear @param body mapa con los datos de la tarea	201 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.
actualizarEstado()	Actualiza el estado de una tarea (pendiente, en_proceso, completada). @param id ID de la tarea a actualizar @param body mapa con los datos de la tarea	200 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.
obtenerStats()	Retorna estadisticas de tareas del usuario agrupadas por estado. @param id ID de la tarea a consultar	200 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.
eliminarTarea()	Elimina una tarea por su ID. @param id ID de la tarea a eliminar @return mensaje de confirmacion	200 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.
editarTarea()	Edita completamente los campos de una tarea existente. @param id ID de la tarea a editar @param request nuevos datos de la tarea	200 OK Permite crear, listar, editar, eliminar y cambiar el estado de las tareas. Tambien expone estadisticas de tareas por usuario.

Configuracion

Class AppConfig @Configuration

Aplicacion Principal

Class TaskflowApplication @SpringBootApplication

Clase principal de la aplicacion TaskFlow CM. Punto de entrada de Spring Boot. Inicializa el contexto de la aplicacion, configura JPA/Hibernate contra PostgreSQL (Neon) y levanta el servidor embebido Tomcat. Extiende SpringBootServletInitializer para soporte de despliegue en WAR externo.

Metodos:

Metodo	Descripcion	Retorna
configure()	Clase principal de la aplicacion TaskFlow CM. Punto de entrada de Spring Boot. Inicializa el contexto de la aplicacion, configura JPA/Hibernate contra PostgreSQL (Neon) y levanta el servidor embebido Tomcat. Extiende SpringBootServletInitializer para soporte de despliegue en WAR externo.	void
main()	Metodo principal que inicia la aplicacion Spring Boot. @param args argumentos de linea de comandos (no requeridos)	void